Programación Concurrente y Distribuida

Tema 4. Monitores

Contexto

- Sistemas basados en memoria compartida con aplicaciones compuestas por procesos que:
  - Acceden a Recursos no compartibles
  - Requieren de Reglas de sincronización
- Problemas
  - Ejecución simultánea => orden parcial + no atomicidad de sentencias de alto nivel => indeterminismo + Recursos no compartibles => Programas incorrectos => Resultados incorrectos
- Necesitamos
  - Garantizar Exclusión Mutua
  - Garantizar las Condiciones de Sincronización

Contexto

- Soluciones basadas en inhibición de interrupciones
  - Sólo son válidas en monoprocesadores
- Soluciones basadas en espera ocupada
  - Ineficientes
- Soluciones basadas en Semáforos
  - Complejas, difíciles de mantener, mecanismo de muy bajo nivel, código de sincronización distribuido entre todos los procesos, etc.
- ¿Monitores?

Definición de Monitor

- Es un mecanismo de abstracción de datos que permite encapsular la representación de recursos no compartibles junto con las operaciones de acceso a los mismos.
- Un monitor tiene cinco componentes
  - Nombre
  - Variables privadas (o permanentes)
  - Operaciones o procedimientos (públicos o privados)
  - Cola de espera
  - Código de inicialización

Ejemplos de sintaxis

```
monitor Ejemplo {
  // Datos privados del monitor
  private variable;
  // Código de inicialización del monitor
  public Ejemplo() {
  }
  // Procedimientos público del monitor
  public void proc1() {
       variable = ...;
  }
}
```

```
{
  // ejemplo es un objeto
  //  de tipo Ejemplo
  ejemplo.proc1();
}
```

```
begin
  ejemplo.proc1();
end;
```

```
Monitor ejemplo;
  var variable;
  export proc1;
  procedure proc1(parámetros);
  var variables locales;
  begin
    código del procedimiento
  end;
  procedure proc2(parámetros);
  var variables locales;
  begin
    código del procedimiento
  end;
begin
  código de inicialización
  del monitor
end;
```

Propiedades

- Las variables del monitor son privadas y el acceso a las mismas sólo es posible a través del conjunto de procedimientos públicos del monitor.
- El monitor es un objeto compartido (elemento pasivo), sus procedimientos son ejecutados por procesos o hilos (elementos activos).
- Ofrece mecanismos de control para la ejecución concurrente de los procedimientos del monitor.
  - Proporciona exclusión mutua en el acceso a las variables.
  - Permite implementar condiciones de sincronización.
- Una vez definido, si es correcto, funcionará igual independientemente del número de procesos que lo usen.

Funcionamiento

- Uso del monitor

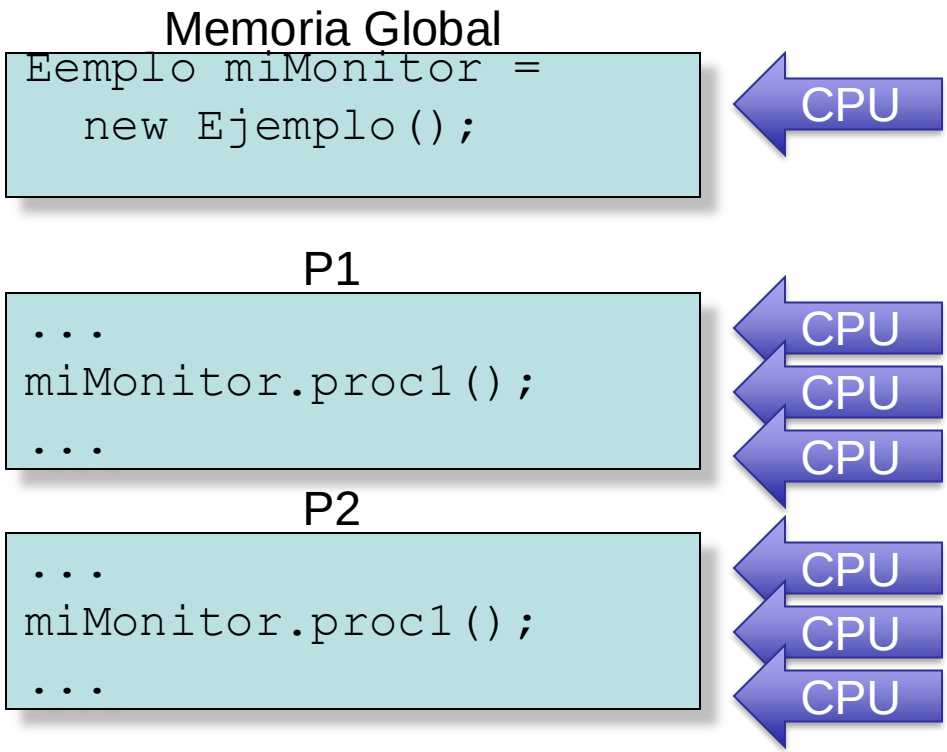
```
nombre.procedimiento(parámetros)
```

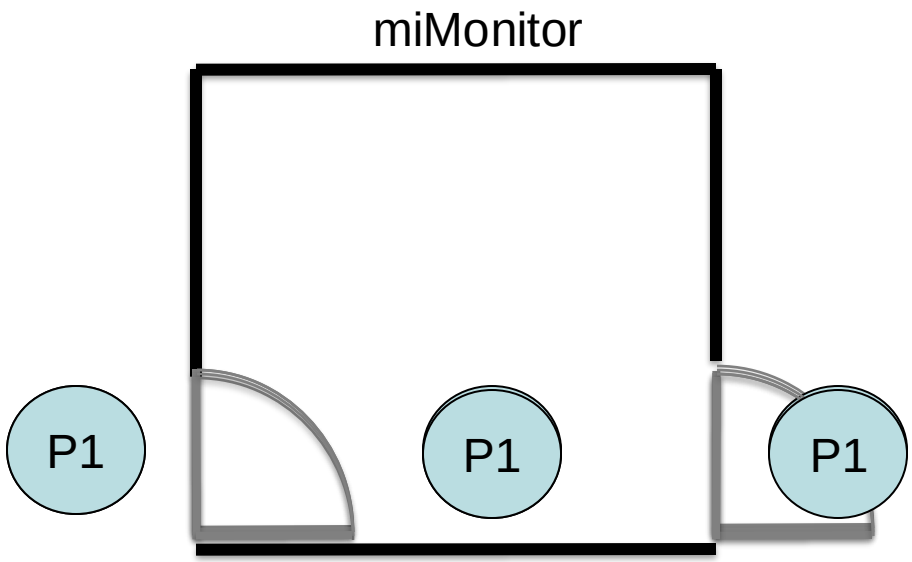
- Funcionamiento
  - Invocar a un procedimiento implica intentar entrar al monitor. Se sale del monitor al terminar el procedimiento o al quedar bloqueado
  - Sólo se puede estar ejecutando **un procedimiento** dentro del monitor en cada momento.
  - Gestión automática de una cola FIFO de espera de procesos que quieren entrar para ejecutar algún procedimiento del monitor mientras éste está en uso.

La ejecución de los procedimientos del monitor se lleva a cabo en exclusión mutua.

Simulación del funcionamiento

- Los procesos van entrando al monitor en el orden en el que intenten ejecutar alguno de los procedimientos del mismo. Sólo uno estará dentro en cada momento.





Exclusión Mutua con Monitores

- Si el recurso está compuesto por una o más variables las declararemos dentro del monitor.
- Si el recurso no es software haremos un monitor que ofrezca las operaciones necesarias para utilizar el recurso.
- Si el número de operaciones a ofrecer es demasiado alto es mejor usar el monitor para implementar un protocolo de entrada y otro de salida que sirva para cualquier sección crítica

  (p.e. Si es un caso de EM de acceso único y exclusivo y sin prioridad pues básicamente simularemos un Semáforo binario)

Exclusión Mutua con Monitores (ejemplo)

- Recurso software protegido: variable **n**

Memoria Global

```
MutexN recurso = new MutexN();
```

Proceso Pi

```
int i = 0;
for ( i = 0 ; i < 10 ; i++ ) {
  recurso.incrementa();
}
```

```
monitor MutexN {
  private int n;

  public MutexN() {
    n = 0;
  }

  public void incrementa() {
    n = n + 1;
  }
}
```

Exclusión Mutua con Monitores

- Recurso hardware protegido: Pantalla (encapsulándola)

Memoria Global

```
MutexPantalla pantalla =
  new MutexPantalla();
```

Luego veremos otra manera de proteger un recurso hardware usando Protocolo de entrada y de salida.

Proceso Uno

```
while (true) {
  pantalla.println("Soy uno");
}
```

Proceso Dos

```
while (true) {
  pantalla.println("Soy dos");
}
```

```
monitor MutexPantalla {
  // Sin datos
  public MutexPantalla() {
    // Sin inicialización
  }
  public void println(String msg) {
    System.out.println(msg);
  }
}
```

Exclusión Mutua con Monitores

- Implementación de semáforo binario con un monitor

```
monitor SemaforoBinario {
  private int valor;
  public SemaforoBinario() {
    valor = 1;
  }
  public void wait() {
    if (valor == 0) BloquearProcesoActual();
    else valor = 0;
  }
  public void signal() {
    if (HayBloqueados()) DesbloquearUno();
    else valor = 1;
  }
}
```

¿Cómo hacer esto?

Memoria Global

```
SemaforoBinario mmutex =
new SemaforoBinario();
int n = 0;
```

Proceso Pi

```
for ( int i = 0 ; i < 10 ; i++ ) {
  mmutex.wait();
  n = n + 1;
  mmutex.signal();
}
```

Condición de Sincronización con Monitores

- Necesitamos bloquear un proceso cuando no se cumple una condición.
- Necesitamos que un proceso activo pueda desbloquear a un proceso bloqueado al hacer que se cumpla la condición que éste esperaba.

Dentro de un monitor se pueden declarar y usar variables de tipo **Condition** que permiten gestionar colas FIFO de procesos bloqueados

Operaciones con variables Condition

- Declaración
  - Variables de tipo **Condition.**
  - **Sólo dentro de un monitor.**
- Usar sólo dentro de los procedimientos del monitor mediante las siguientes operaciones:
  - **delay(c)**: Bloquea al proceso que lo ejecuta liberando previamente la exclusión mutua del monitor.
  - **resume(c)**: Si hay algún proceso bloqueado en esta condición desbloquea uno (FIFO), si no **no hace nada**.
  - **empty(c)**: Devuelve verdadero si NO hay ningún proceso bloqueado en esta cola.

Exclusión Mutua con Monitores

- Recurso hardware protegido: Pantalla (con PE/PS)

Memoria Global

```
MutexPantalla pantalla =
  new MutexPantalla();
```

Proceso Uno

```
while (true) {
  pantalla.PedirAcceso();
  System.out.println("Soy uno");
  pantalla.LiberarAcceso();
}
```

Proceso Dos

```
while (true) {
  pantalla.PedirAcceso();
  System.out.format("%2$s %1$s",
       "dos","Soy");
  pantalla.LiberarAcceso();
}
```

```
monitor MutexPantalla {
  private boolean ocupada;
  private Condition c;

  public MutexPantalla() {
    ocupada=false;
  }
  public void PedirAcceso() {
    if (ocupada) delay(c);
    ocupada=true;
  }
  public void LiberarAcceso(){
    ocupada=false;
    resume(c);
  }
}
```

Sincronización con monitores

- Los procesos se comunican/sincronizan a través de los datos protegidos por el monitor.
- Una variable **Condition** por condición lógica.
- Las distintas condiciones serán relativas a los datos privados del monitor o el estado del recurso representado para garantizar el acceso en exclusión mutua.
- Los distintos procesos ejecutan procedimientos del monitor, procedimientos en los que podrán quedar bloqueados o desbloquear a otros procesos.

NUNCA USAR DELAY DE FORMA INCONDICIONAL

Sincronización con monitores

Memoria Global

```
Sincro sincro = new Sincro();
```

Proceso Hola

```
System.out.println("Hola");
sincro.informar();
```

Proceso Mundo

```
Sincro.esperar();
System.out.println("Mundo");
```

```
monitor Sincro {
  private boolean escrito;
  private Condition c;

  public Sincro() {
    escrito = false;
  }

  public void esperar() {
    if (!escrito) delay(c);
  }

  public void informar() {
    escrito = true;
    resume(c);
  }
}
```

El problema de `resume`

```
monitor Ejemplo{
  private Condition c1;
  private Condition c2;
  public void proc1() {
    delay(c1); // MALA IDEA
    // resto
  }
  public void proc2() {
    resume(c1);
    // resto
  }
}
```

CPU

P1

```
...
monitor.proc1();
...
```

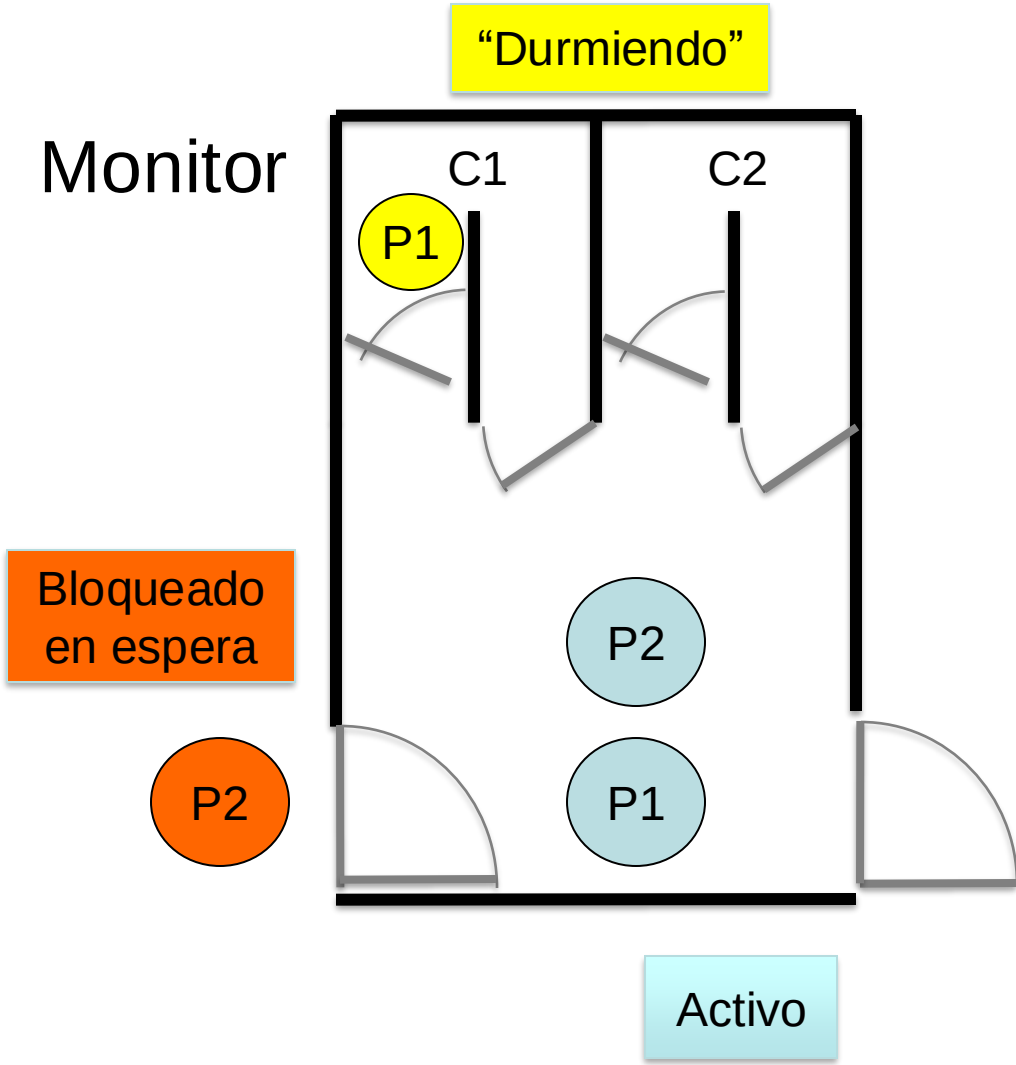
CPU

P2

```
...
monitor.proc2();
...
```

CPU

¿Dos procesos pueden ejecutar código del monitor simultáneamente?

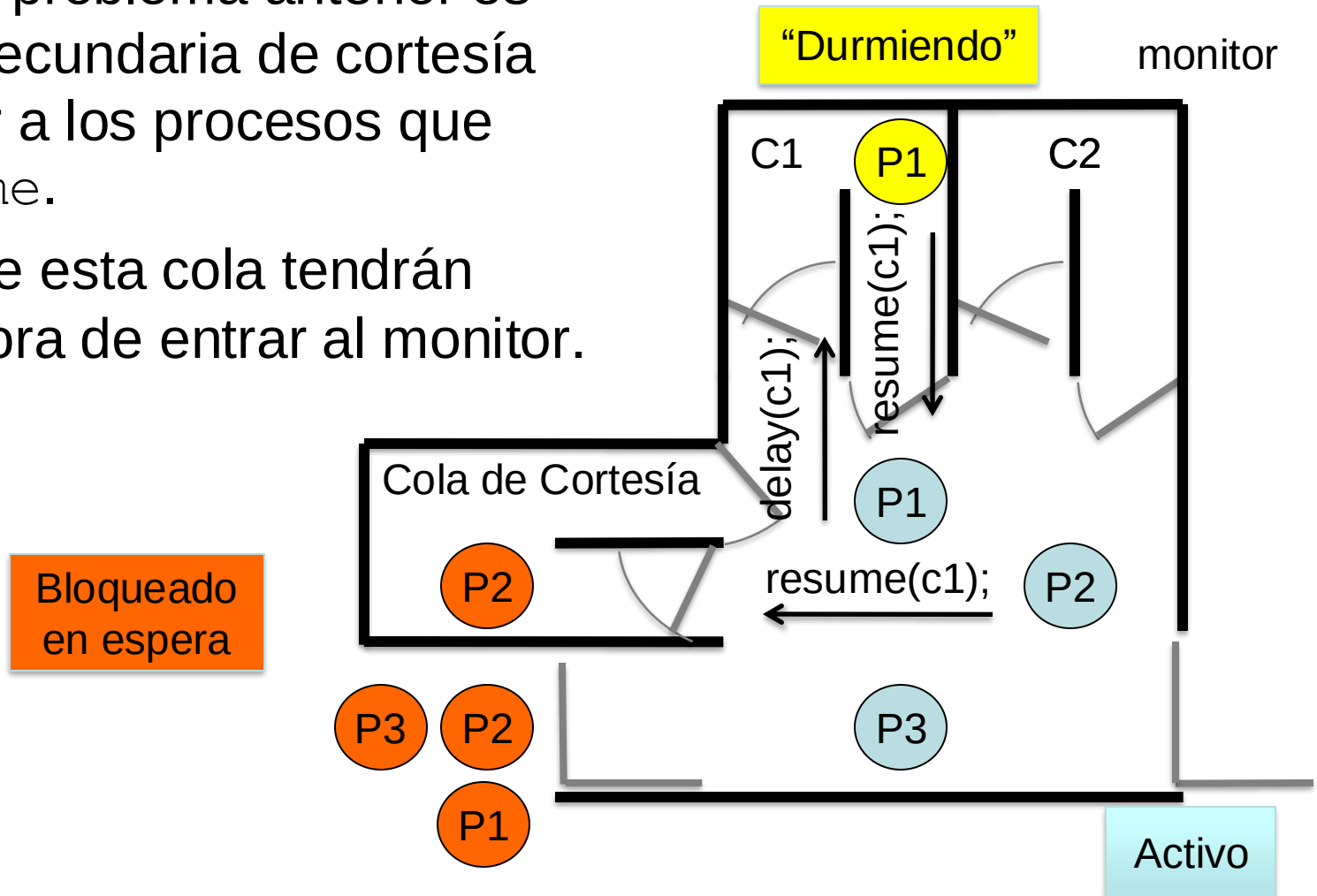


Soluciones al problema de `resume`

- Desbloquear y Salir
  - El que desbloquea cede la exclusión mutua del monitor al desbloqueado
    - Desbloquear y retorno forzado
    - Desbloquear y esperar
    - Desbloquear y espera urgente
- Desbloquear y Continuar
  - El desbloqueado se ejecutará después de que el desbloqueador salga del monitor

La cola de cortesía

- Solución de tipo desbloquear y salir. (Desbloqueo y espera urgente)
- Una solución al problema anterior es usar una cola secundaria de cortesía donde bloquear a los procesos que ejecuten `resume`.
- Los procesos de esta cola tendrán prioridad a la hora de entrar al monitor.



Exclusión Mutua con Monitores

- Implementación de semáforo binario con monitor

```
monitor SemaforoBinario {
  private int valor;
  private Condition cola;
  public SemaforoBinario() {
    valor = 1;
  }
  public void wait() {
    if (valor == 0) delay(cola);
    else valor = 0;
  }
  public void signal() {
    if (!empty(cola)) resume(cola);
    else valor = 1;
  }
}
```

La llamada a:
empty(cola),
es necesaria

Memoria Global

```
SemaforoBinario mmutex =
new SemaforoBinario();
int n = 0;
```

Proceso Pi

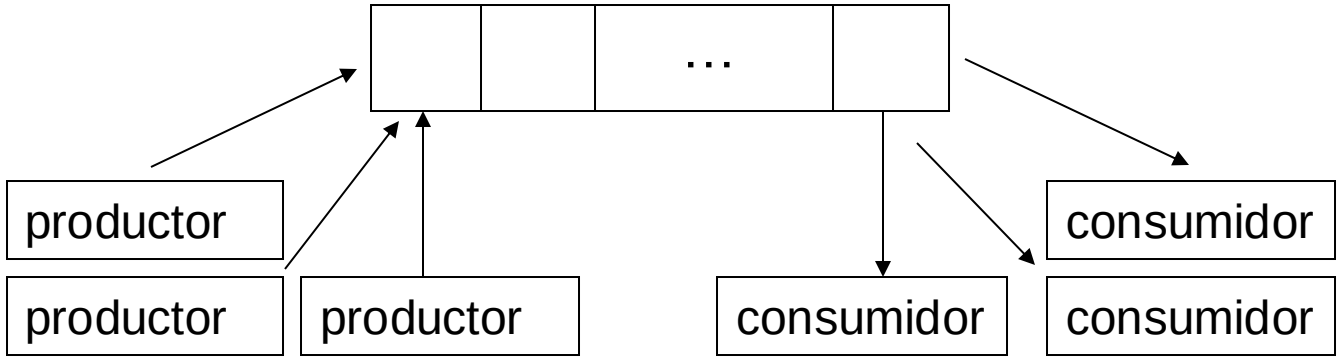
```
for ( int i = 0 ; i < 10 ; i++ ) {
  mmutex.wait();
  n = n + 1;
  mmutex.signal();
}
```

Problemas Clásicos

- Productor/Consumidor
- Lectores y Escritores
- Cena de los Filósofos

Productor**es** / Consumidor**es**

- Procesos que generan información (productores) que es utilizada por otros procesos (consumidores).
- Posible diferencia de velocidad de ejecución requiere usar un **buffer compartido (cola)** para desacoplar velocidades.
- Necesidad de acceder en exclusión mutua al buffer.



- El código de inserción y de extracción de elementos en la cola pasa a ser una **Sección Crítica.**

Productor / Consumidor

Esquema básico

Proceso Productor

```
while (true) {
  // Producir elemento
  Elemento e = new Elemento();
  // Insertar elemento

}
```

Proceso Consumidor

```
while (true) {
  // Extraer elemento
  Elemento e = ...
  // Consumir elemento
  e.usa();
}
```

Análisis Productor/Consumidor

- Recursos no compartibles
  - Cola
  - Se insertan y se extraen elementos
- Condiciones de sincronización
  - Sólo se inserta si hay espacio
  - Sólo se extrae si hay elementos

- Cola será una variable definida dentro de un monitor
- Habrá dos procedimientos: insertar y extraer
- Dos variables **Condition**, una para cada condición lógica:
  - nolleno
  - novacio

Productores/Consumidores

```
monitor Buffer {
  private Cola cola;
  private Condition novacio;
  private Condition nolleno;

  public Buffer(int n) {
    cola = new Cola(n);
  }
  public void inserta(Elemento e) {
    if (cola.llena()) delay(nolleno);
    cola.push(e);
    resume(novacio);
  }
  public Elemento extrae() {
    if (cola.vacia()) delay(novacio);
    Elemento e = cola.pop();
    resume(nolleno);
    return e;
  }
}
```

Proceso Productor

```
while (true) {
  // Producir elemento
  Elemento e = new Elemento();
  // Insertar elemento
  buffer.inserta(e);
}
```

Memoria Global

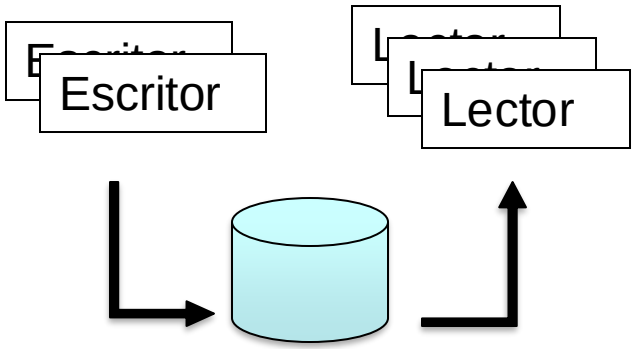
```
Buffer buffer = new Buffer(5);
```

Proceso Consumidor

```
while (true) {
  // Extraer elemento
  Elemento e = buffer.extrae();
  // Consumir elemento
  e.usa();
}
```

Lectores / Escritores

- Procesos que acceden a un recurso en modo lectura (lectores) y procesos que acceden en modo escritura al mismo recurso (escritores).



Recurso:
- BBDD
- WEB
- Disco Duro

- **Prioridad lectores:** ningún lector debe esperar salvo que un escritor esté escribiendo. Un lector no espera por el simple hecho de que haya un escritor esperando.
- **Prioridad escritores:** cuando un escritor espera, debe escribir lo antes posible. Es decir, ningún nuevo lector debe iniciar su lectura si hay algún escritor esperando.

Lectores / Escritores

Esquema básico

Memoria Global

```
Recurso recurso;
```

Proceso Tipo Lector

```
while (true) {
  // Protocolo Entrada Lector
  LEER DEL RECURSO
  // Protocolo Salida Lector
}
```

Proceso Tipo Escritor

```
while (true) {
  //Protocolo entrada Escritor
  ESCRIBIR EN EL RECURSO
  // Protocolo salida Escritor
}
```

Análisis Prioridad Lectores

- Recursos no compartibles
  - **nl**, **escribiendo**
  - Recurso (sólo a veces)
- Condiciones de sincronización
  - Sólo se lee si no se está escribiendo
  - Sólo se escribe si no se está escribiendo ni leyendo

- **nl** y **escribiendo** variables del monitor
- El recurso no puede estar dentro del monitor (o solo podría leer uno a la vez)
- Habrá dos procedimientos para cada tipo de proceso. Protocolos de entrada y salida.
- Dos variables **Condition**, una para cada condición:
  - leer
  - escribir

Lectores/Escritores

prioridad lectores v2.0

```
monitor Acceso {
  private int nl;
  private boolean escribiendo;
  private Condition escribir;
  private Condition leer;

  public Acceso() {
    nl = 0;
    escribiendo = false;
  }
  ...
}
```

```
public void abrir_lectura() {
  if (escribiendo) delay(leer);
  nl++;
  resume(leer);
}

public void cerrar_lectura() {
  nl--;
  if (nl==0) resume(escribir);
}
```

```
public void abrir_escritura() {
  if (escribiendo || nl > 0)
    delay(escribir);
  escribiendo = true;
}

public void cerrar_escritura() {
  escribiendo = false;
  if (!empty(leer))
    resume(leer);
  else
    resume(escribir);
}
```

Lectores / Escritores

Esquema básico

Memoria Global

```
Recurso recurso;
Acceso acceso;
```

Proceso Tipo Lector

```
while (true) {
  // Protocolo Entrada Lector
  acceso.abrir_lectura();
  LEER DEL RECURSO
  // Protocolo Salida Lector
  acceso.cerrar_lectura();
}
```

Proceso Tipo Escritor

```
while (true) {
  //Protocolo entrada Escritor
  acceso.abrir_escritura();
  ESCRIBIR EN EL RECURSO
  // Protocolo salida Escritor
  acceso.cerrar_escritura();
}
```

Análisis Prioridad Escritores

- Recursos no compartibles
  - **nl, nle, nee**, **escribiendo**
  - Recurso (sólo a veces)
- Condiciones de sincronización
  - Sólo se lee si no se está escribiendo ni hay escritores esperando
  - Sólo se escribe si no se está escribiendo ni leyendo

- **nl, nle, nee** y **escribiendo** variables del monitor
- El recurso no puede estar dentro del monitor

- Habrá dos procedimientos para cada tipo de proceso. Protocolos de entrada y salida.
- Dos variables **Condition**, una para cada condición:
  - leer
  - escribir

Lectores/Escritores

prioridad escritores (v1.0)

```
monitor Acceso {
  private int nl, nee, nle;
  private boolean escribiendo;
  private Condition escribir;
  private Condition leer;

  public Acceso() {
    nl = 0; nle = 0; nee = 0;
    escribiendo = false;
  }
  ...
```

```
public void abrir_lectura() {
  if (escribiendo || nee > 0) {
    nle++;
    delay(leer);
    nle--;
  }
  nl++;
  resume(leer);
}
public void cerrar_lectura() {
  nl--;
  if (nl==0) resume(escribir);
}
```

```
public void abrir_escritura() {
  if (escribiendo || nl > 0) {
    nee++;
    delay(escribir);
    nee--;
  }
  escribiendo = true;
}

public void cerrar_escritura() {
  escribiendo = false;
  if (!empty(escribir))
    resume(escribir);
  else
    resume(leer);
}
```

Lectores/Escritores

prioridad escritores (v2.0)

```
monitor Acceso {
  private int nl;
  private boolean escribiendo;
  private Condition escribir;
  private Condition leer;

  public Acceso() {
    nl = 0;
    escribiendo = false;
  }
  ...
```

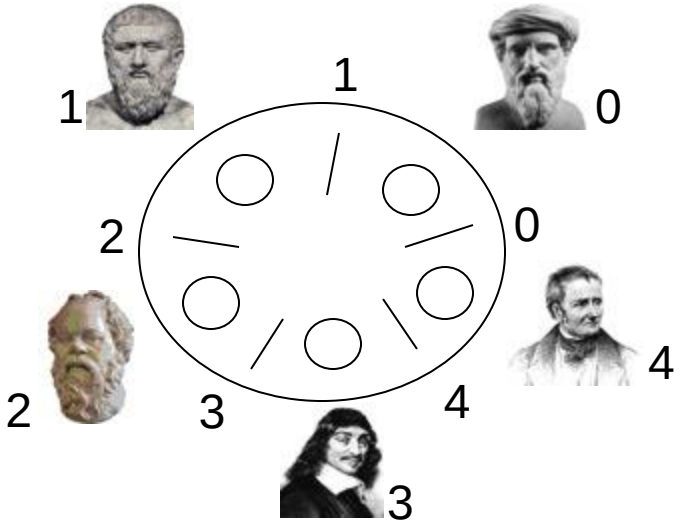
```
public void abrir_lectura() {
  if (escribiendo ||
      !empty(escribir)) {
    delay(leer);
  }
  nl++;
  resume(leer);
}
public void cerrar_lectura() {
  nl--;
  if (nl==0) resume(escribir);
}
```

```
public void abrir_escritura() {
  if (escribiendo || nl > 0) {
    delay(escribir);
  }
  escribiendo = true;
}

public void cerrar_escritura() {
  escribiendo = false;
  if (!empty(escribir))
    resume(escribir);
  else
    resume(leer);
}
```

Filósofos

- Cinco filósofos se dedican a pensar y comer de forma alterna. Para comer usan una mesa con cinco sillas, cinco palillos y cinco platos.
- Cuando un filósofo decide comer intenta coger sus dos palillos y sólo comerá cuando tenga los dos.



- Procesos que necesitan utilizar varios recursos no compartibles simultáneamente. El número de recursos no es suficiente para abastecer a todos los procesos al mismo tiempo.

Análisis Filósofos

- Recursos no compartibles
  - **palillos o asientos**

- **estado de los palillos o de los filósofos** como variables del monitor

- Condiciones de sincronización
  - Filósofo **i** sólo come si sus dos palillos están libres
  - Dejar de comer implica la posible liberación de dos filósofos distintos **i-1** e **i+1**

- Habrá dos procedimientos uno para sentarse y otro para levantarse.
- Una variable **Condition** por filósofo
  - esperando[i]

Filósofos

Esquema básico

Proceso Tipo Filósofo i

```
while (true) {
  // PENSAR
  mesa.sentarse(i);
  // COMER
  mesa.levantarse(i);
}
```

Memoria Global

```
Mesa mesa;
```

Comida de Filósofos

```
monitor Mesa {
  private Condition esperando[N];
  private String estado[N];

  public Mesa() {
    for ( int i = 0 ; i < N ; i++ ) estado[i] = “pensando”;
  }

  public void sentarse(int filosofo) {
    estado[filosofo] = “hambriento”;
    if (!puede_comer(filosofo)) delay(esperando[filosofo]);
    estado[filosofo] = “comiendo”;
  }

  public void levantarse(int filosofo) {
    estado[filosofo] = “pensando”;
    int siguiente = (filosofo+1) % N;
    int anterior = (filosofo+N-1) % N;
    if (puede_comer(siguiente)) resume(esperando[siguiente]);
    if (puede_comer(anterior))  resume(esperando[anterior]);
  }
  ...
}
```

Método puede_comer

```
monitor Mesa {
  ...
  private boolean puede_comer(int filosofo) {
    int siguiente = (filosofo+1) % N;
    int anterior = (filosofo+N-1) % N;
    if ((estado[anterior] != “comiendo”) &&
        (estado[siguiente] != “comiendo”) &&
        (estado[filosofo] == “hambriento”)) {
      return true;
    } else {
      return false;
    }
  }
}
```

Tipos de resume

- Desbloquear y Retorno Forzado (DR)
  - Tras desbloquear se realiza un return.
- Desbloquear y Esperar (DE)
  - Tras desbloquear se bloquea al proceso actual en la cola de entrada al monitor. Se cede la exclusión mutua al desbloqueado.
- Desbloquear y Esperar con Urgencia (DU)
  - Tras desbloquear se bloquea al proceso en una cola especial con prioridad sobre la de entrada al monitor.

- Desbloquear y continuar (DC)
  - El proceso desbloqueado podrá ejecutarse inmediatamente después de que el proceso actual salga del monitor (pero no sabemos cuándo lo hará).

Diferencias según el tipo de resume

```
monitor Test{
  Condition c1, c2;

  public void a() {
    delay(c1);
  }

  public void b() {
    delay(c2);
    resume(c1);
    resume(c1);
  }

  public void c() {
    resume(c2);
  }
}
```

P1	P2
`test.a();`	`test.b();`

P3	P4
`test.c();`	`test.c();`

Cola Entrada: P1, P2, P3, P4.
C1:
C2:
Cola Cortesía:
Dentro Monitor:

DU	DC	DR	DE
P1 DEL(C1)	P1 DEL(C1)	P1 DEL(C1)	P1 DEL(C1)
P2 DEL(C2)	P2 DEL(C2)	P2 DEL(C2)	P2 DEL(C2)
P3 RES(C2)	P3 RES(C2)	P3 RES(C2)	P3 RES(C2)
P2 RES(C1)	**P3 END**	**P3 END**	P2 RES(C1)
P1 END	P2 RES(C1)	P2 RES(C1)	**P1 END**
P3 END	P2 RES(C1)	**P2 END**	P4 RES(C2)
P2 RES(C1)	**P2 END**	**P1 END**	**P4 END**
P2 END	**P1 END**	P4 RES(C2)	**P3 END**
P4 RES(C2)	P4 RES(C2)	**P4 END**	P2 RES(C1)
P4 END	**P4 END**		**P2 END**

Llamadas anidadas (¡Cuidado!)

```
monitor A {
  public void metodo1() {
    b.metodo2();
  }
  public void metodo3() {
    ...
  }
}
```

```
monitor B {
  private Condition c;
  public void metodo2() {
    if (...) delay(c);
  }
  public void metodo4() {
    resume(c);
  }
}
```

Proceso Tipo 1

```
while (true) {
  a.metodo1();
}
```

Proceso Tipo 2

```
while (true) {
  a.metodo3();
  b.metodo4();
}
```

Memoria Global

```
A a = new A();
B b = new B();
```